Lecture 20 (MIP Introduction)

May 3, 2026

Today we will briefly discuss Mixed Integer Programming. We will not go too in depth, but we will talk about how to formulate some problems as MIPs. In the notebook (on Canvas), you will also see a simple example of an MIP implemented on the solver.

Definitions

A **Mixed Integer Program (MIP)** is any optimization problem that can written in the following form:

$$\text{minimize } \vec{c} \cdot \vec{x} + \vec{c} \cdot \vec{z}$$

s.t.

$$A\vec{x} + B\vec{z} \leq \vec{b}$$

and

$$\vec{x} \in \mathbb{R}^{n_1}_{\geq 0},\ \vec{z} \in \mathbb{Z}^{n_2}_{\geq 0}$$

Here, we are taking $\mathbb{R}_{\geq 0}$ and $\mathbb{Z}_{\geq 0}$ to be the set of non-negative real numbers and non-negative integers respectively. This definition is pretty general. There are two common special cases we see often.

A **Pure Integer Program (IP)** is any optimization problem that can written in the following form:

$$\text{minimize } \vec{c} \cdot \vec{z}$$

s.t.

$$B\vec{z} \leq \vec{b}$$

and

$$\vec{z} \in \mathbb{Z}^{n}_{\geq 0}$$

A **Binary Integer Program (BIP)** is any optimization problem that can written in the following form:

$$\text{minimize } \vec{c} \cdot \vec{z}$$

s.t.

$$B\vec{z} \leq \vec{b}$$

and

$$z_i \in \{0, 1\}\ \forall i$$

Notes

1. We have a standard form for convenience of proofs and definitions. Many people will give different and equivalent definitions. Transforming between these is more or less the same as we did for LPs.

2. When formulating an MIP, write it in the form that is easiest to understand/communicate. Don't convert it to standard form unless it is more clear. The solver will do this for you, and understanding the model is usually more important when communicating/implementing the model.

3. In practice, solving MIPs is a large and active field of research. We will not focus on how to solve these. If you want to learn more, I encourage you to take Bala Krishnamoorthy's Math 567 class (Integer and Combinatorial Optimization). If you are unable to take this class (it is only offered every other Spring semester), I encourage you to read through his notes on his website. They are very well written.

Some Examples

Let's give a few examples of BIP formulations of problems. When formulating problems, it is usually best to start by determining what the decision variables (DVs) are first. After this, the constraint and objective will follow.

Example 1: Scheduling

Statement

Suppose you have to schedule n employees to n shifts. Each employee must work exactly one shift, and each shift must have exactly one employee. The cost of scheduling employee i to shift j is c_{ij}. Find the schedule that minimizes the total cost.

Formulation

First, start with the DVs.

$$x_{ij} = \begin{cases} 1 & \text{If employee } i \text{ is scheduled for shift } j \\ 0 & \text{Otherwise} \end{cases}$$

Now, for the constraints. First the constraint that each employee must work exactly one shift:

$$\sum_j x_{ij} = 1$$

and now for the constraint that each shift can have at most one employee:

$$\sum_i x_{ij} = 1$$

Now, for the objective:

$$\text{mininize } \sum_i \sum_j c_{ij} x_{ij}$$

If we want to write this in a more standard format, we would say we have the BIP below:

$$\text{mininize } \sum_i \sum_j c_{ij} x_{ij}$$

s.t.

$$\sum_j x_{ij} = 1$$

$$\sum_i x_{ij} = 1$$

$$x_{ij} \in \{0, 1\} \ \forall i, j$$

This is a basic scheduling problem. In practice, many scheduling problems have additional constraints and may be a little bit more general.

Example 2: Knapsack

Normally the knapsack problem is stated a little differently. I like the following statement.

Statement

Suppose an adventurer gets to the bottom of a dungeon and finds a dragon's hoard. Inside of the hoard there are various gems of differing weights and values. The adventurer can only carry at most, N pounds of gems. Further, the adventurer cannot cut any gems (gem cutting equipment is heavy, and the adventurer cannot take them into a dungeon). If gem i has a weight of w_i and a value of v_i, find the collection of gems that maximizes the value of the gems the adventurer leaves with.

Formulation

First start with the D.V.s.

$$x_i = \begin{cases} 1 & \text{If the adventurer takes gem } i \text{ out of the dungeon} \\ 0 & \text{Otherwise} \end{cases}$$

Now for the constraints. We must obey the total weight constraint:

$$\sum_i w_i x_i \leq N$$

Now, for the objective:

$$\text{maximize } \sum_i v_i x_i$$

If we want to write this in a more standard format, we would say we have the BIP below:

$$\text{maximize } \sum_i v_i x_i$$

s.t.

$$\sum_i w_i x_i \leq N$$

$$x_i \in \{0, 1\}$$

Example 3: Sudoku

Statement

In sudoku, the goal is to fill in the empty squares of a 9×9 grid such that each row, each column, and each 3×3 box contains each integer $[1, 9]$ exactly once. A picture of a typical sudoku puzzle is shown below.

4	8	6		5		9		
1	9					5		7
				1	4		2	8
	7	8		3			5	
		5	1	4		8	6	
	4				6			2
8	6	7	4				3	1
		4	8		3	7		
3			6		1			

Formulation

We need to be careful here. The numbers are placeholders. The numbers in each cell could be replaced with colors and the logic of the puzzle still holds. As such, we will define the following decision variables:

$$x_{ijk} = \begin{cases} 1 & \text{If the cell in row } i \text{ column } j \text{ contains a value of } k \\ 0 & \text{Otherwise} \end{cases}$$

Now, for the constraints. The first constraint is subtle. We need to ensure each cell contains exactly one number. This is not explicitly stated in the problem statement, but is required for the decision variables we defined:

$$\sum_k x_{ijk} = 1 \ \forall i, j$$

Now, we need to ensure that each row contains exactly one of each number:

$$\sum_j x_{ijk} = 1 \ \forall i, k$$

and we also need to ensure that each column contains exactly one of each number:

$$\sum_i x_{ijk} = 1 \ \forall j, k$$

Now, we need to deal with the box constraints. These are a little bit more involved. We will consider the top left box, then translate the cells appropriately for the other boxes. In practice, you could do nested for loops to create these in code.

$$1 = x_{1+n,1+m,k} + x_{2+n,1+m,k} + x_{3+n,1+m,k} + x_{1+n,2+m,k} + x_{2+n,2+m,k} + x_{3+n,2+m,k} + x_{1+n,3+m,k} + x_{2+n,3+m,k} + x_{3+n,3+m,k}$$

for all k and for all $n, m \in \{0, 3, 6\}$. Note that the n, m values dictate the box. If you want to write this more concisely, we can do a double sum:

$$1 = \sum_{a=1}^{3} \sum_{b=1}^{3} x_{a+n,b+m,k} \ \forall k, \ \forall n, m \in \{0, 3, 6\}$$

We don't need an objective. A valid puzzle is a valid puzzle. Some solvers work without an objective. If you need an objective for the solver, the easiest work around is to create a constant objective:

$$\text{minimize } 0$$

Combining these yields the formulation below:

$$\text{minimize } 0$$

s.t.

$$\sum_{k} x_{ijk} = 1 \ \forall i, j$$

$$\sum_{j} x_{ijk} = 1 \ \forall i, k$$

$$\sum_{i} x_{ijk} = 1 \ \forall j, k$$

$$1 = \sum_{a=1}^{3} \sum_{b=1}^{3} x_{a+n,b+m,k} \ \forall k, \ \forall n, m \in \{0, 3, 6\}$$

and for

$$x_{ijk} \in \{0, 1\} \ \ 1 \leq i, j, k \leq 9$$

Formal Logic and BIP

Suppose $L_1, L_2, \ldots, L_n$ are a collection of logical statements. Each can either be true or false. We will use the symbols "$\wedge$" to mean "and" and "$\vee$" to mean "or" and "$\neg$" to mean "not". If binary variable x_i corresponds to logical statement L_i, then we can represent logical statements involving "and", "or" and "not" statements as BIP constraints.

Statement		Constraint
$L_1 \wedge L_2$	$\rightarrow$	$x_1 + x_2 = 2$
$L_1 \wedge L_2 \wedge \cdots \wedge L_n$	$\rightarrow$	$\sum_{i=1}^{n} x_i = n$
$L_1 \vee L_2$	$\rightarrow$	$x_1 + x_2 \geq 1$
$L_1 \vee L_2 \vee \cdots \vee L_n$	$\rightarrow$	$\sum_{i=1}^{n} x_i \geq 1$
$\neg L_1$	$\rightarrow$	$1 - x_1 \geq 1$

We can chain together more complicated expressions into more complicated constraints:

$$(L_1 \vee L_2) \wedge (\neg L_3)$$

can be broken into separate constraints:

$$L_1 \vee L_2 \rightarrow x_1 + x_2 \geq 1$$

$$\neg L_3 \rightarrow 1 - x_3 \geq 1$$

Since these are broken by "ands", it is easiest to break it into two constraints. This gives rise to **Conjunctive Normal Form (CNF)**.

CNF

Every logical statement can be written in the form

$$L_1 \wedge L_2 \wedge \cdots \wedge L_n$$

where each L_i contains only "or" and "not" statements. A statement in this form is in CNF. Each clause (Note that $L_1, L_2, \ldots, L_n$ are all clauses since they do not contain any "and" statements) will correspond to a separate constraint.